《AI 秘书 – 基于大模型的智能个人助理》
需求分析文档（Requirements Specification）

软件工程课程项目组

2026 年 4 月 20 日

目录

1 引言 **3**
1.1 项目背景与痛点洞察 3
1.2 术语与缩略语 3
1.3 文档约定 3

2 涉众分析（Stakeholder Analysis） **4**
2.1 涉众识别与利益诉求 4
2.2 用户画像与典型场景 4

3 功能需求分析（Functional Requirements） **5**
3.1 FR-01 用户身份与会话管理体系 5
3.2 FR-02 NLP 日程引擎（Schedule Action） 5
3.3 FR-03 结构化学习规划（Study Action） 6
3.4 FR-04 心理卫士独立通道（Psych Action） 6
3.5 FR-05 智能对话与上下文记忆（Chat Action） 6
3.6 FR-06 端云双态灾备机制 7

4 非功能需求（Non-Functional Requirements） **7**
4.1 性能需求（Performance Requirements） 7
4.2 可用性与可靠性需求（Availability & Reliability） 7
4.3 安全性需求（Security Requirements） 8
4.4 易用性需求（Usability Requirements） 8
4.5 可维护性与可扩展性需求 8

5 数据字典概述（Data Dictionary） **9**
5.1 核心实体定义 9
5.2 Action 类型枚举 9

5.3 关键状态流转 10

6 项目管理与团队协作契约 10

1 引言

1.1 项目背景与痛点洞察

在移动互联网向万物互联演进的时代背景下，传统端侧应用普遍存在以下结构性痛点：

1. **功能入口深不可测**：用户完成一项核心任务（如创建日程、记录心情）往往需要在多级菜单中反复跳转，认知负荷极高。
2. **意图识别响应迟缓**：现有方案过度依赖云端大模型的单次往返调用，在弱网或高并发场景下延迟陡增，用户体验断崖式下跌。
3. **强依赖网络缺乏灾备**：绝大多数 AI 助手在无网络环境下完全瘫痪，无法提供任何本地智能服务，可用性边界极为脆弱。
4. **数据隐私与权限越界**：过度申请系统权限、缺乏本地化的物理级操作拦截，导致用户敏感信息暴露在不可控的第三方链路中。

本项目旨在面向 **HarmonyOS 生态**，利用原生 ArkTS 构建一款**端云协同的轻量级 Agent 架构**个人助理。通过引入上海交通大学 GLM-5 大模型作为云端逻辑中枢，结合强大的本地正则引擎、relationalStore 持久化存储与多层安全探针，实现一套**高可用**、**低延迟**、**隐私优先**的个人智能助理体系。

1.2 术语与缩略语

术语	英文全称	释义
Agent	Intelligent Agent	具备感知、决策与执行能力的智能代理实体
RDB	Relational Database	关系型数据库，此处特指 HarmonyOS relationalStore
Prompt	Prompt Engineering	通过结构化模板引导大模型生成特定输出的工程技术
Mock	Mock Engine	本地模拟引擎，用于云端不可用时提供降级服务
Regex	Regular Expression	正则表达式，用于自然语言中的实体提取
Action	Action Router	动作路由，系统对用户需求进行分类后的执行通道

表 1: 核心术语表

1.3 文档约定

本文档采用以下优先级约定描述需求：

- **必须（Must）**：系统上线不可或缺的硬性约束，缺失将导致核心功能不可用。

- **应当（Should）**：强烈建议实现的功能或性能指标，缺失会显著降低用户体验。
- **可以（May）**：锦上添花型能力，在时间与资源允许时迭代补充。

2 涉众分析（Stakeholder Analysis）

2.1 涉众识别与利益诉求

本项目的目标涉众可分为以下四类，其利益诉求直接决定了功能边界与质量属性的权重分配：

涉众类别	代表角色	核心利益诉求
终端用户	高校学生、职场新人	通过自然语言快速完成日程创建、学习规划与情绪记录；无网时仍可本地使用
系统运维方	课程项目组自身	模块化架构便于 4 人并行开发；Mock 挡板测试降低联调成本
平台审核方	应用市场/课程评委	权限最小化、数据本地脱敏、无过度网络请求
未来开发者	开源社区/后续迭代组	接口契约清晰、文档完备、可扩展的 Action 插件机制

表 2: 涉众利益诉求矩阵

2.2 用户画像与典型场景

用户画像 A：考研学生”小张”

- 场景 1：输入”帮我安排明天下午 3 点在图书馆复习数学”→ 系统自动解析时间地点事件 → 弹窗确认 → 写入日程。
- 场景 2：无网环境下输入”我想学 Python”→ Mock 引擎识别”学习”特征词 → 返回本地内置的 Python 学习路径。
- 场景 3：考前焦虑时输入”最近压力好大，睡不着”→ 系统触发 psych 通道 → 跳转紫色心理辅导页 → 情绪分级监测 → 给出舒缓建议。

用户画像 B：职场新人”小李”

- 场景 1：登录后进入会话中心 → 创建”工作沟通”会话 → 记录多项目并行时的碎片化任务。
- 场景 2：输入”下周三上午 9 点去客户公司汇报”→ Regex 引擎识别”下周三””上午 9 点””客户公司”→ 结构化日程。

3 功能需求分析（Functional Requirements）

系统核心功能划分为六大矩阵，采用**读写执行体解耦**的模块化设计思想。

3.1 FR-01 用户身份与会话管理体系

优先级：Must

1. **用户注册与登录**：支持本地用户名/密码注册，密码长度不少于 4 位；登录成功后持久化会话状态，下次启动自动跳转会话中心。
2. **多会话生命周期管理**：用户可创建多个**普通对话**（general）和独立**心理辅导会话**（psych），每个会话拥有独立的 Messages 与 Schedules 数据隔离域。
3. **会话级联删除**：删除普通会话时，系统必须级联清理其关联的全部 Messages、Schedules 以及挂载的 Psych 子会话，避免数据孤岛。
4. **智能会话标题生成**：系统根据用户首条输入内容，通过本地规则引擎自动分类并生成精炼标题（如”日程安排 · 下周三汇报”、” 心理：压力调节”），分类体系涵盖：日程安排、学习计划、工作沟通、出行安排、生活规划、情绪倾诉、压力调节、关系困扰、情绪支持。

3.2 FR-02 NLP 日程引擎（Schedule Action）

优先级：Must

1. **自然语言意图识别**：LLMService 将用户输入归类为 schedule 动作，返回结构化 JSON。
2. **纯本地 Regex 实体提取管线**：WriteExecutor 通过手写正则引擎从自然语言中极速剥离：
   - **日期**：支持” 今天/明天/后天/大后天”、” 下周三”、”5 月 20 日”、”2026-04-18” 等多元表达。
   - **时间**：支持” 下午 3 点”、”15:30”、” 三点半”、” 一刻”、中文数字等复杂语义。
   - **地点**：通过” 在 XX 开会/去 XX” 模式提取地点实体。
   - **事件**：经多轮噪声清洗（去除” 帮我/安排/提醒我” 等语气词）提取核心事件。
3. **双路合并补强机制**：页面层将模型返回的结构化结果与用户原话再次进行本地解析合并，避免模型漏掉” 明天/周三/下午三点” 等关键信息。
4. **物理级确认与落盘**：日程写入前强制唤起 ActionDialog，用户点击” 确认添加” 后，由单例 DatabaseManager 写入 relationalStore。
5. **日程全生命周期管理**：支持查看、编辑（带日期滚轮 + 时间滚轮选择器）、删除、标记完成。

3.3 FR-03 结构化学习规划（Study Action）

优先级：Should

1. **主题识别与路径推荐**：系统识别”学习 Python””复习数据结构”等意图后，由 WriteExecutor 返回内置的离线学习推荐。
2. **内置技能图谱**：覆盖 Python（2 周初级）、数据结构与算法（3 周中级）、HarmonyOS ArkTS（2 周初级）、英语（4 周）等结构化路径，每条路径包含资源列表与阶段计划。
3. **零网络依赖**：学习推荐完全基于本地规则引擎，无需调用云端 API，确保任何网络环境下均可使用。

3.4 FR-04 心理卫士独立通道（Psych Action）

优先级：Must

1. **独立会话物理隔离**：心理辅导会话拥有独立的 Conversations 记录、紫色主题 UI(#7C3AED)、以及与普通会话完全隔离的消息存储。
2. **情绪分级监测体系**：
   - **High 级**：检测到”崩溃/绝望/活不下去/自杀/自残”等极端词汇 → 触发高危预警，建议联系专业援助。
   - **Medium 级**：检测到”焦虑/压力/难过/失眠/痛苦”→ 建议情绪记录与短时休息。
   - **Low 级**：检测到”累/烦/低落/郁闷”→ 鼓励继续倾诉或简单复盘。
3. **温和陪伴 Prompt**：PsychChat 页面在调用 LLM 时，自动追加”请用心理支持和温和陪伴的方式回应”的系统指令。
4. **安全探针智能放行**：SecurityChecker 在 psych 场景下识别到”联系/求助/专业/帮助/热线”等正面支持性词汇时，不做拦截，避免误杀救援信息。

3.5 FR-05 智能对话与上下文记忆（Chat Action）

优先级：Must

1. **非对称气泡排版**：左侧 AI 消息采用蓝色头像（#3B82F6）+ 白色气泡，右侧用户消息采用绿色头像（#10B981）+ 深绿气泡（#2D6A4F），媲美微信级沉浸式体验。
2. **短期记忆注入**：ReadExecutor 自动回溯最近 5 条对话历史，拼装进 Prompt，保证多轮对话的上下文连贯性。
3. **底部 Tab 导航**：主页面提供”对话/通知/我的”三栏 Tab 切换，对话 Tab 承载聊天与日程面板，通知 Tab 展示待完成日程统计，我的 Tab 展示用户信息与会话切换入口。

3.6 FR-06 端云双态灾备机制

优先级：Must

1. **云端优先模式**：默认通过 HTTP POST 调用上海交通大学 GLM-5 大模型 API（models.sjtu.edu.cn），temperature 设为 0.4 以保证输出稳定性。
2. **本地 Mock 优雅降级**：当检测到以下任一条件时，系统在 $<$ 500ms 内自动切换至本地 Mock 引擎：
   - 网络连接类型为 NONE（物理断网）
   - HTTP 响应码非 200
   - 请求超时（connectTimeout / readTimeout 超过 60s）
   - 返回内容 JSON 解析失败
3. **特征词拦截路由**：Mock 引擎基于本地规则库对输入进行特征匹配：" 日程/会议/安排"→schedule；" 学习/课程/Python"→study；" 焦虑/累/烦"→psych；其他 →chat。

4 非功能需求（Non-Functional Requirements）

4.1 性能需求（Performance Requirements）

指标编号	指标名称	目标值	验证方式
NFR-P01	Mock 模式端到端响应延迟	$\leq$ 500ms	在无网环境下使用秒表/日志打点测量
NFR-P02	Regex 实体提取耗时	$\leq$ 5ms	对 100 条测试语料进行循环压测取均值
NFR-P03	数据库查询分页上限	50 条/次	代码静态检查 predicates.limitAs(50)
NFR-P04	冷启动数据库初始化耗时	$\leq$ 300ms	清空数据后首次进入 App 计时
NFR-P05	UI 列表滚动帧率	$\geq$ 55fps	ArkUI Inspector 性能面板观察
NFR-P06	内存峰值占用	$\leq$ 180MB	DevEco Profiler 内存采样

表 3: 性能需求指标表

4.2 可用性与可靠性需求（Availability & Reliability）

1. **无网环境可用率**：四大核心 Action（chat/schedule/study/psych）在无网环境下必须 100% 可用，降级切换成功率不得低于 99.5%。
2. **数据库事务可靠性**：所有写操作（saveMessage/saveSchedule/updateSchedule）必须通过 relationalStore 的原子操作保证，异常时回滚且不破坏数据一致性。
3. **异常熔断机制**：LLMService 在云端调用失败时不得抛出未捕获异常导致 App 崩溃，必须优雅降级至 Mock 并返回友好提示。

4. **会话状态持久化**：用户退出登录前，当前会话 ID、当前用户 ID 必须通过 AppSettings 表持久化，下次启动时自动恢复现场。

4.3 安全性需求（Security Requirements）

1. **权限最小化原则**：module.json5 中仅申请 ohos.permission.INTERNET 与 ohos.permission.GET_NETWO 禁止申请与功能无关的敏感权限（如位置、通讯录、相机）。

2. **五层本地安全探针**：

   - **L1 危险指令拦截**：rm -rf / shutdown / 删除系统 / 执行脚本 / 下载文件 / 访问链接
   - **L2 有害内容拦截**：自杀方法 / 自残方法 / 杀人 / 暴力教程（psych 场景下智能放行救援性内容）
   - **L3 隐私泄露拦截**：密码 / 银行卡 / 身份证 / 验证码 + 风险动词（发送/提供/上传/泄露）组合检测
   - **L4 提示词注入拦截**：” 忽略以上”/” 绕过安全”/” 无需确认”/”system prompt”
   - **L5 越权动作拦截**：delete / modify / share / purchase / transfer / payment / execute 等动作类型直接阻断

3. **物理级操作确认**：所有标记为 sensitiveActions 的操作（schedule/delete/modify/share/purchase）必须在用户点击确认按钮后方可执行数据库写入。

4. **数据本地脱敏与沙箱隔离**：所有用户数据（聊天记录、日程、密码）仅存储在应用私有 relationalStore 中，不经过任何外部网络传输。

4.4 易用性需求（Usability Requirements）

1. **自然语言即界面**：用户无需学习任何命令语法，直接以日常口语与系统交互。

2. **容错性输入**：系统对” 帮我安排一下明天下午三点去会议室开会吧” 和” 明天下午 3 点会议室开会” 应产生一致的解析结果。

3. **视觉一致性**：普通会话采用绿色系主题(#2D6A4F),心理辅导采用紫色系主题(#7C3AED),通过色彩编码降低用户认知成本。

4. **即时反馈**：AI 思考时显示”AI 正在思考”Loading 动画，操作成功/失败均有 Toast 或气泡消息反馈。

4.5 可维护性与可扩展性需求

1. **模块化挡板测试**：各模块通过标准化 JSON 契约通信，开发阶段可基于 Mock 数据独立测试，降低联调成本。

2. **Action 插件预留**：WriteExecutor 的多态路由设计预留了新增 Action 类型的扩展位，后续可低成本接入天气查询、翻译等新能力。

3. **数据库版本兼容**：DatabaseManager 通过 safeAddColumn 机制实现表结构向前兼容，旧版本数据可平滑升级。

5 数据字典概述（Data Dictionary）

5.1 核心实体定义

实体名称	主键	核心属性说明
User（用户）	id (INTEGER)	username(唯一), password, createdTime
Conversation（会话）	id (INTEGER)	userId(外键), title(智能生成), type(general/psych), parentConversationId, updatedTime, lastMessage
Message（消息）	id (INTEGER)	conversationId(外键), content, isUser(布尔), timestamp(ISO8601)
Schedule（日程）	id (INTEGER)	conversationId(外键), content(结构化文本), createdTime, remindTime(可选), isDone(布尔)
AppSetting（设置）	key (TEXT)	value(文本), 用于存储 currentUserId/currentConversationId 等运行时状态

表 4: 核心实体数据字典

5.2 Action 类型枚举

Action 值	业务含义	下游处理逻辑
chat	普通闲聊	直通 ArkUI 渲染层，非对称气泡展示
schedule	日程安排	Regex 实体提取 →ActionDialog 确认 →relationalStore 落盘
study	学习建议	本地规则引擎匹配内置技能图谱 → 结构化路径返回
psych	心理辅导	情绪分级分析 → 跳转 PsychChat 紫色独立页面
unknown	未识别	降级为 chat，返回友好兜底回复

表 5: Action 类型枚举表

5.3 关键状态流转

会话状态机（Conversation State Machine）：

```
状态集 S = { CREATED, ACTIVE, ARCHIVED, DELETED }
初始状态: CREATED
转移函数:
  CREATED  --(用户打开会话)--> ACTIVE
  ACTIVE   --(用户发送消息)--> ACTIVE  (更新updatedTime)
  ACTIVE   --(用户点击删除)--> DELETED (级联清理Messages/Schedules)
  ACTIVE   --(长时间无操作)--> ARCHIVED (预留, 当前版本未实现)
```

Listing 1: 会话生命周期状态流转伪代码

LLM 调用状态机：

```
状态集 S = { CLOUD_READY, MOCK_FALLBACK, ERROR_RECOVER }
初始状态: CLOUD_READY
转移函数:
  CLOUD_READY    --(HTTP 200 + 合法JSON)--> 正常返回
  CLOUD_READY    --(网络异常/超时/非200)--> MOCK_FALLBACK
  CLOUD_READY    --(JSON解析失败)--> MOCK_FALLBACK
  MOCK_FALLBACK  --(特征词匹配成功)--> 返回标准化JSON
  MOCK_FALLBACK  --(特征词匹配失败)--> 默认chat兜底
```

Listing 2: 双态调度状态流转伪代码

6 项目管理与团队协作契约

本项目由 4 名组员在 4 周内完成，核心代码 100% 自研。遵循**接口契约优先于实现**的协作原则，各模块通过标准化 JSON 接口通信，禁止跨层直接调用 UI 或数据库。

Git 协作规范：提交前缀严格遵循 `[feat]` / `[fix]` / `[refactor]` / `[test]` / `[docs]`，配合标准化 Mock 数据进行挡板测试，保障 4 人并行协作零冲突。